
列举必做内容样例。

样例 1（局部公共子表达式消除）：

输入：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #2
7  v3 := t3
8  t4 := v3 + #1
9  v4 := t4
10 t5 := v1 + #3
11 v5 := t5
12 t6 := v1 + v2
13 v6 := t6
14 t7 := v1 + #2
15 v7 := t7
16 t8 := v1 + #2
17 v2 := t8
18 t9 := #0
19 RETURN t9
```

输出：

对于这段只包含单个基本块的中间代码，对基本块中表达式构造有向无环图，并且运用代数恒等式转换。我们检查有向无环图中一个节点 **N**，是否与另一个节点 **M** 具有相同的运算符与子节点，若相同则消除公共子表达式，生成的中间代码可以是这样的：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #2
7  v3 := t3
8  t4 := v3 + #1
9  v4 := t4
10 t5 := v1 + #3
11 v5 := t5
12 t6 := v1 + v2
13 v6 := t6
14 v7 := t3
15 v2 := t3
16 t9 := #0
17 RETURN t9
```

如果你的方法足够聪明，你会发现 **t4** 与 **t5** 也是相同的，通过设计合适的框架，能够使得公共子表达式的消除效果更好。

样例 2（局部无用代码消除）：

输入：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  v3 := #2
7  v4 := #3
8  t3 := v2 + v3
9  v1 := t3
10 t4 := v1 - v4
11 v2 := t4
12 t5 := v2 + v3
13 v3 := t5
14 t6 := v1 - v4
15 v4 := t6
16 t7 := v1 + v3
17 v5 := t7
18 t8 := v1 - v4
19 v6 := t8
20 t9 := v4 + v6
21 v7 := t9
22 t10 := v2 + v1
23 v1 := t10
24 t11 := v7 + v6
25 v8 := t11
26 WRITE v8
27 t12 := v1 + v4
28 v9 := t12
29 WRITE v9
30 t13 := #0
31 RETURN t13
```

输出：

对于 v1-v7 七个变量，构造有向无环图。我们基于有向无环图，迭代地去除没有父节点且未被使用的根节点，消除无用代码，因此生成的中间代码可以是这样的：

```
1  FUNCTION main :
2  READ t1
3  READ t2
4  v2 := t2
5  v3 := #2
6  v4 := #3
7  t3 := v2 + v3
8  v1 := t3
9  t4 := v1 - v4
10 v2 := t4
11 t5:= v2 + v3
12 v3 := t5
13 t6 := v1 - v4
14 v4 := t6
15 t7 := v1 - v4
16 v6 := t7
17 t8 := v4 + v6
```

```
18  v7 := t8
19  t9 := v2 + v1
20  v1 := t9
21  t10 := v7 + v6
22  v8 := t10
23  WRITE v8
24  t11 := v1 + v4
25  v9 := t11
26  WRITE v9
27  t12 := #0
28  RETURN t12
```

样例3（常量折叠）：

输入：

```
1  FUNCTION main :
2  DEC v6 40
3  v1 := #30
4  t1 := v1 / #5
5  v2 := t1
6  t2 := #9 - v2
7  v2 := t2
8  t3 := v2 * #4
9  v3 := t3
10 t4 := #2 * v3
11 t5 := v1 - t4
12 v4 := t5
13 t6 := v4 * #4
14 t7 := &v6 + t6
15 t8 := *t7
16 v5 := t8
17 RETURN #0
```

输出：

在编译时，我们使用常量折叠技术，识别并计算常量表达式，以避免在运行时计算他们的值。

通常，我们迭代式地遍历中间代码，基于复制传播及常量传播技术，用常量代替一切识别为常量的变量或是表达式。对于以上的中间代码，优化后中间代码可以是这样的：

```
1  FUNCTION main :
2  DEC v6 40
3  v1 := #30
4  t1 := #6
5  v2 := #6
6  t2 := #3
7  v2 := #3
8  t3 := #12
9  v3 := #12
10 t4 := #24
11 t5 := #6
12 v4 := #6
13 t6 := #24
14 t7 := &v6 + #24
15 t8 := *t7
```

```
16  v5 := t8
17  RETURN #0
```

6.3.7 样例（选做要求）

这节列举选做要求样例。

样例 1（全局公共子表达式消除）：

输入：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v2 + #2
9  v4 := t4
10 t5 := v1 + v2
11 v5 := t5
12 t6 := v5 - v3
13 v6 := t6
14 t7 := v1 + v3
15 v7 := t7
16 t8 := v1 + v6
17 v4 := t8
18 v8 := #0
19 LABEL label1 :
20 IF v8 < v1 GOTO label2
21 GOTO label3
22 LABEL label2 :
23 t9 := v8 + #1
24 v8 := t9
25 IF v1 < v2 GOTO label4
26 GOTO label5
27 LABEL label4 :
28 t10 := v1 + v2
29 v8 := t10
30 t11 := v1 + #1
31 v6 := t11
32 GOTO label6
33 LABEL label5 :
34 t12 := v2 + v3
35 v1 := t12
36 t13 := v1 + v2
37 v5 := t13
38 LABEL label6 :
39 GOTO label1
40 LABEL label3 :
41 RETURN #0
```

输出：

对于全局的公共子表达式的消除，通常我们采用可用表达式模式，分析到达某一程序点 p 的

所有路径上，是否存在可用的表达式。全局公共子表达式消除后，中间代码可以是这样的：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v2 + #2
9  v4 := t4
10 t5 := v1 + v2
11 v5 := t5
12 t6 := v5 - v3
13 v6 := t6
14 t7 := v1 + v3
15 v7 := t7
16 t8 := v1 + v6
17 v4 := t8
18 v8 := #0
19 LABEL label1 :
20 IF v8 < v1 GOTO label2
21 GOTO label3
22 LABEL label2 :
23 t9 := v8 + #1
24 v8 := t9
25 IF v1 < v2 GOTO label4
26 GOTO label5
27 LABEL label4 :
28 v8 := v5
29 t10 := v1 + #1
30 v6 := t10
31 GOTO label6
32 LABEL label5 :
33 t11 := v2 + v3
34 v1 := t11
35 t12 := v1 + v2
36 v5 := t12
37 LABEL label6 :
38 GOTO label1
39 LABEL label3 :
40 RETURN #0
```

样例 2（常量传播）：

输入：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  v4 := #2
9  t4 := v3 * v4
10 v5 := t4
```

```

11  t5 := #2 + v3
12  v8 := t5
13  t6 := v3 * #2
14  v7 := t6
15  t7 := #2 * v3
16  t8 := t7 + #4
17  v9 := t8
18  IF v9 == v7 GOTO label2
19  LABEL label1 :
20  v3 := #3
21  t9 := v4 + #5
22  v5 := t9
23  t10 := #2 * v8
24  v1 := t10
25  t11 := v9 - v1
26  v2 := t11
27  GOTO label3
28  LABEL label2 :
29  v2 := v3
30  t12 := v5 + v2
31  v1 := t12
32  t13 := v7 * v3
33  v9 := t13
34  LABEL label3 :
35  t14 := v1 + v2
36  v8 := t14
37  v7 := v3
38  t15 := v5 - v2
39  v9 := t15
40  RETURN #0

```

输出:

对于全局常量传播的计算，在实践中，我们仅要求读者能够熟练使用简单常量传播算法。在

常量替换后，中间代码可以是这样的:

```

1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  v4 := #2
9  t4 := v3 * #2
10 v5 := t4
11 t5 := #2 + v3
12 v8 := t5
13 t6 := v3 * #2
14 v7 := t6
15 t7 := #2 * v3
16 t8 := t7 + #4
17 v9 := t8
18 IF v9 == v7 GOTO label2
19 LABEL label1 :
20 v3 := #3
21 v5 := #7
22 t9 := #2 * v8

```

```

23  v1 := t9
24  t10 := v9 - v1
25  v2 := t10
26  GOTO label1
27  LABEL label2 :
28  v2 := v3
29  t11 := v5 + v2
30  v1 := t11
31  t12 := v7 * v3
32  v9 := t12
33  LABEL label3 :
34  t13 := v1 + v2
35  v8 := t13
36  v7 := v3
37  t14 := v5 - v2
38  v9 := t14
39  RETURN #0

```

如果我们使用本书上介绍的条件常量传播算法，并且采取更加智能的常量传播框架，可以发现，以上的中间代码中， $v9 \neq v7$ 恒成立，程序只会执行 `label1` 分支的语句。因此你可以将中间代码优化为如下的形式。

包含变量的表达式存在不同的情况，在简化时需注意（例如 $a+1$ 与 $a-1$ ， $a+1 > a-1$ 并不恒成立）。

```

1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  v4 := #2
9  t4 := v3 * #2
10 v5 := t4
11 t5 := #2 + v3
12 v8 := t5
13 t6 := v3 * #2
14 v7 := t6
15 t7 := #2 * v3
16 t8 := t7 + #4
17 v9 := t8
18 v3 := #3
19 v5 := #7
20 t9 := #2 * v8
21 v1 := t9
22 v2 := #0
23 v8 := v1
24 v7 := v3
25 v9 := #7
26 RETURN #0

```

样例 3（全局无用代码消除）：

输入：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v2 + #2
9  v4 := t4
10 t5 := v1 + v2
11 v5 := t5
12 v6 := #0
13 LABEL label1 :
14 IF v6 < v1 GOTO label2
15 GOTO label3
16 LABEL label2 :
17 v1 := v5
18 t6 := v1 - #1
19 v4 := t6
20 IF v3 < v4 GOTO label4
21 GOTO label5
22 LABEL label4 :
23 v3 := #4
24 v2 := v4
25 t7 := v1 + v2
26 v6 := t7
27 GOTO label6
28 LABEL label5 :
29 t8 := v3 + #3
30 v3 := t8
31 v6 := v1
32 t9 := v1 + #1
33 v6 := t9
34 LABEL label6 :
35 t10 := v6 + #3
36 v6 := t10
37 GOTO label1
38 LABEL label3 :
39 t11 := #2 * v1
40 v5 := t11
41 t12 := v2 + v5
42 v6 := t12
43 RETURN #0
```

输出：

对于一个变量在某个程序点上定义的值，若在之后的程序执行过程中未被使用，该定义语句是无用代码，可被削减。同时，对于不被执行的条件分支中的语句，也被认为是无用代码。对于无用代码消除，我们通常采取活跃变量分析的模式。生成的中间代码可以是这样的：

```
1  FUNCTION main :
2  READ t1
```

```

3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v1 + v2
9  v5 := t4
10 v6 := #0
11 LABEL label1 :
12 IF v6 < v1 GOTO label2
13 GOTO label3
14 LABEL label2 :
15 v1 := v5
16 t5 := v1 - #1
17 v4 := t5
18 IF v3 < v4 GOTO label4
19 GOTO label5
20 LABEL label4 :
21 v3 := #4
22 v2 := v4
23 GOTO label6
24 LABEL label5 :
25 t6 := v3 + #3
26 v3 := t6
27 t7 := v1 + #1
28 v6 := t7
29 LABEL label6 :
30 t8 := v6 + #3
31 v6 := t8
32 GOTO label1
33 LABEL label3 :
34 t9 := #2 * v1
35 v5 := t9
36 t10 := v2 + v5
37 v6 := t10
38 RETURN #0

```

样例 4（循环不变代码外提）：

输入：

```

1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v2 * #2
9  v4 := t4
10 t5 := v3 * v4
11 v5 := t5
12 t6 := v5 * v4
13 t7 := v1 + t6
14 v6 := t7

```

```

15  t8 := v3 - v1
16  v7 := t8
17  LABEL label1 :
18  v1 := v7
19  t9 := v7 * v4
20  v2 := t9
21  IF v3 > v4 GOTO label2
22  GOTO label3
23  LABEL label2 :
24  v3 := #4
25  t10 := v1 * #5
26  v4 := t10
27  GOTO label4
28  LABEL label3 :
29  t11 := v1 - #3
30  v8 := t11
31  LABEL label4 :
32  IF v1 > v2 GOTO label1
33  GOTO label5
34  LABEL label5 :
35  t12 := #2 * v4
36  v9 := t12
37  RETURN #0

```

输出:

程序执行的过程中，大部分的执行时间都花在循环上，因此，研究人员针对循环设计了许多编译优化技术。对于循环内部不管执行了多少次，都能得到相同结果的表达式，我们可以将其移动到循环外部，以避免循环过程中反复执行该表达式。经过循环不变代码外提后，中间代码可以是这样的：

```

1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := v1 + #1
7  v3 := t3
8  t4 := v2 * #2
9  v4 := t4
10 t5 := v3 * v4
11 v5 := t5
12 t6 := v5 * v4
13 t7 := v1 + t6
14 v6 := t7
15 t8 := v3 - v1
16 v7 := t8
17 v1 := v7
18 LABEL label1 :
19 t9 := v7 * v4
20 v2 := t9
21 IF v3 > v4 GOTO label2
22 GOTO label3

```

```
23 LABEL label2 :
24 v3 := #4
25 t10 := v1 * #5
26 v4 := t10
27 GOTO label4
28 LABEL label3 :
29 t11 := v1 - #3
30 v8 := t11
31 LABEL label4 :
32 IF v1 > v2 GOTO label1
33 GOTO label5
34 LABEL label5 :
35 t12 := #2 * v4
36 v9 := t12
37 RETURN #0
```

样例 5（强度削减）：

输入：

```
1 FUNCTION main :
2 READ t1
3 v1 := t1
4 READ t2
5 v2 := t2
6 t3 := #4 * v1
7 v3 := t3
8 t4 := #4 * v2
9 v4 := t4
10 t5 := v1 - #1
11 v5 := t5
12 t6 := v2 * #2
13 v6 := t6
14 t7 := v1 + v2
15 v7 := t7
16 t8 := v3 * v5
17 v8 := t8
18 t9 := v7 - v4
19 v9 := t9
20 LABEL label1 :
21 IF v3 < v9 GOTO label2
22 GOTO label3
23 LABEL label2 :
24 t10 := v1 + #1
25 v1 := t10
26 t11 := #4 * v1
27 v3 := t11
28 t12 := v3 * v4
29 v5 := t12
30 LABEL label4 :
31 IF v4 > v5 GOTO label5
32 GOTO label6
33 LABEL label5 :
34 t13 := v2 - #1
35 v2 := t13
36 t14 := #4 * v2
37 v4 := t14
```

```
38 GOTO label4
39 LABEL label6 :
40 GOTO label1
41 LABEL label3 :
42 RETURN #0
```

输出:

对于循环内部的昂贵操作，我们可以对其做一些语义相同的转化，将“昂贵”的操作转化为相对便宜的操作。我们运用强度削减，对其中可以简化的乘法操作转化为加法或减法操作，生成的中间代码可以是这样的：

```
1  FUNCTION main :
2  READ t1
3  v1 := t1
4  READ t2
5  v2 := t2
6  t3 := #4 * v1
7  v3 := t3
8  t4 := #4 * v2
9  v4 := t4
10 t5 := v1 - #1
11 v5 := t5
12 t6 := v2 * #2
13 v6 := t6
14 t7 := v1 + v2
15 v7 := t7
16 t8 := v3 * v5
17 v8 := t8
18 t9 := v7 - v4
19 v9 := t9
20 LABEL label1 :
21 IF v3 < v9 GOTO label2
22 GOTO label3
23 LABEL label2 :
24 t10 := v1 + #1
25 v1 := t10
26 t11 := v3 + #4
27 v3 := t11
28 t12 := v3 * v4
29 v5 := t12
30 LABEL label4 :
31 IF v4 > v5 GOTO label5
32 GOTO label6
33 LABEL label5 :
34 t13 := v2 - #1
35 v2 := t13
36 t14 := v4 - #4
37 v4 := t14
38 GOTO label4
39 LABEL label6 :
40 GOTO label1
41 LABEL label3 :
42 RETURN #0
```